



Resolvedor de Laberintos Gigantes (Proyecto Final)

Integrantes:

Roberto Antonio Lopez Garcia

Esmil Adames

Oscar Disla

Mirelys De La Rosa

Materia:

Programación Paralela

Profesor:

Erick Leonardo Perez Veloz

Índice:

Introducción:	3
Justificación del tema elegido:	4
Objetivos:	4
Descripción del problema:	5
Contexto del problema seleccionado:	5
Aplicación del problema en un escenario real:	5
Importancia del paralelismo en la solución:	5
Cumplimiento de los requisitos del proyecto:	6
Diseño de la solución:	7
Arquitectura general del sistema:	7
Diagrama de componentes:	7
Estrategia de paralelización utilizada:	8
Herramientas y tecnologías empleadas:	8
Implementación técnica:	9
Estructura del proyecto:	9
Explicación del código clave:	9
Uso de mecanismos de sincronización:	10
Justificación técnica de las decisiones tomadas:	10
Evaluación de desempeño:	10
Prueba de integración — laberinto de 10x10:	10
Comparativa a gran escala:	11
Análisis de cuellos de botella y limitaciones:	11
Descripción del reparto de tareas:	12
Herramientas utilizadas para la coordinación:	12
Conclusiones:	13
Principales aprendizajes técnicos:	13
Retos enfrentados y superados:	13
Referencias:	14
Anexos:	14
Manual de ejecución del sistema:	14
Pruebas complementarias:	14
Enlace al repositorio:	14

Introducción:

Este documento describe el diseño, la implementación y los resultados del proyecto Resolvedor de Laberintos Gigantes, desarrollado para la asignatura de Programación Paralela. El sistema genera laberintos de tamaño configurable y calcula el camino más corto entre su entrada y su salida, usando cuatro algoritmos de búsqueda distintos: BFS, DFS, A* y Dijkstra. Los cuatro se ejecutan al mismo tiempo sobre el mismo laberinto, aprovechando Task Parallel Library (TPL) para repartir el trabajo entre varios núcleos del procesador.

El objetivo del proyecto no es solo resolver un laberinto, sino demostrar con datos reales cuánto se gana en tiempo al ejecutar varias tareas al mismo tiempo, en comparación con ejecutarlas una detrás de otra. Para eso, el propio programa corre las dos versiones sobre el mismo laberinto: primero la secuencial, y después la paralela, y calcula cuánto más rápida fue la segunda y qué tan bien se aprovecharon los núcleos usados.

Justificación del tema elegido:

El equipo evaluó varios temas antes de decidirse por este. La ventaja de un laberinto frente a otros escenarios (por ejemplo, sistemas de vuelos o ambulancias) es que su tamaño no depende de ningún conjunto de datos externo: se puede fijar libremente para representar la carga de trabajo que el proyecto necesite demostrar. Una cuadrícula de mil por mil casillas equivale a un millón de posiciones, y cada uno de los cuatro algoritmos tiene que revisar vecinos, marcar celdas visitadas y calcular distancias en cada una de ellas — es una cantidad de trabajo real, no artificial, y además fácil de visualizar y explicar. A la vez, el problema conserva la misma estructura que enfrentan sistemas de uso cotidiano, como los que calculan rutas sobre mapas de ciudades.

Objetivos:

El objetivo general del proyecto es diseñar e implementar un sistema que genere laberintos de gran tamaño y resuelva el camino más corto utilizando múltiples algoritmos de búsqueda en paralelo, midiendo y comparando su desempeño frente a una ejecución secuencial equivalente.

De ese objetivo general se desprenden los siguientes objetivos específicos:

- Generar laberintos grandes de forma automática, garantizando que siempre tengan solución, mediante un algoritmo de backtracking aleatorio.
- Implementar cuatro algoritmos de búsqueda —BFS, DFS, A* y Dijkstra— bajo un contrato común que permita tratarlos de forma intercambiable.
- Ejecutar los cuatro algoritmos de forma simultánea mediante Task Parallel Library, compartiendo la misma estructura de datos en memoria sin condiciones de carrera.
- Comparar el tiempo de ejecución, el camino encontrado y la cantidad de nodos explorados entre los cuatro algoritmos.
- Medir el speedup y la eficiencia obtenidos al pasar de una ejecución secuencial a una paralela, en distintos tamaños de laberinto.
- Permitir que el tamaño del laberinto se configure como parámetro de entrada, sin necesidad de modificar el código fuente.

Descripción del problema:

Contexto del problema seleccionado

Un laberinto de N por N casillas puede entenderse como un grafo: cada celda es un nodo, y existe una conexión hacia cada celda vecina con la que no comparte una pared. Resolver el laberinto —encontrar el camino más corto entre la entrada y la salida— es, en el fondo, un problema de búsqueda sobre ese grafo. Lo que cambia entre BFS, DFS, A^* y Dijkstra no es el problema que resuelven, sino la estrategia que usan para recorrerlo: cuál vecino visitan primero, qué información adicional utilizan para decidir por dónde avanzar, y qué garantías ofrecen sobre la calidad del camino que encuentran.

La escala es lo que convierte este ejercicio en un problema de cómputo real y no solo un ejercicio de algoritmos. Un laberinto de mil casillas por lado tiene un millón de nodos; uno de diez mil casillas por lado tiene cien millones. En ambos casos, cada algoritmo tiene que recorrer una fracción significativa de esos nodos antes de llegar a la salida, y hacerlo cuatro veces —una por algoritmo— de forma secuencial multiplica considerablemente el tiempo total de espera.

Aplicación del problema en un escenario real

El mismo principio que resuelve este proyecto a pequeña escala es el que sostiene, a gran escala, a los sistemas de navegación GPS y a los motores de videojuegos: ambos recorren grafos grandes —el mapa de calles de una ciudad, el escenario de un juego— para encontrar rutas óptimas entre dos puntos. El grafo de calles de una ciudad como Santo Domingo tiene fácilmente varios cientos de miles de intersecciones y tramos de calle, y un sistema de navegación no se limita a calcular una sola ruta: suele evaluar varias alternativas al mismo tiempo —la más rápida, la más corta, la que evita peajes— antes de decidir cuál mostrarle al usuario. Ese es exactamente el mismo patrón que este proyecto reproduce a una escala manejable: varios algoritmos explorando el mismo grafo en paralelo, cada uno con su propia estrategia.

Importancia del paralelismo en la solución

Si los cuatro algoritmos corrieran uno después del otro, el tiempo total de la comparativa sería la suma de los cuatro tiempos individuales, aun cuando ninguno depende del resultado de los demás. Ejecutarlos en paralelo permite que el tiempo total se acerque al tiempo del algoritmo más lento de los cuatro, en lugar de a la suma de todos. En laberintos grandes, donde cada algoritmo puede tardar un tiempo apreciable, esa diferencia se vuelve significativa y es, además, la que este proyecto mide de forma explícita a través del speedup y la eficiencia calculados en cada corrida.

Cumplimiento de los requisitos del proyecto:

La siguiente tabla resume cómo cada uno de los requisitos planteados por la asignatura se ve reflejado en una parte concreta del sistema implementado.

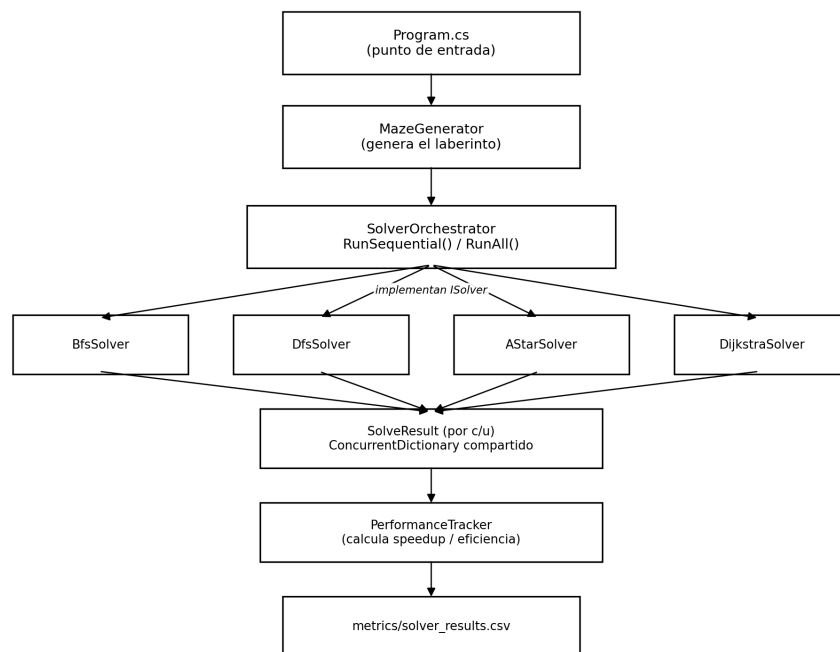
Criterio	Cómo lo aborda el proyecto
Ejecución simultánea de múltiples tareas	SolverOrchestrator.RunAll() lanza los cuatro algoritmos como tareas independientes mediante Task.Run, y las cuatro corren al mismo tiempo sobre el mismo laberinto.
Necesidad de compartir datos entre tareas	Todas las tareas leen la misma matriz de celdas, generada una única vez, y cada una escribe su resultado en una estructura compartida y segura para concurrencia (ConcurrentDictionary).
Exploración de diferentes estrategias de paralelización	Cada algoritmo recorre el grafo de forma distinta —anchura, profundidad, heurística, costo garantizado—, de modo que las cuatro tareas paralelas representan patrones de exploración genuinamente distintos, no cuatro copias del mismo trabajo.
Escalabilidad con más recursos	El tamaño del laberinto se recibe como parámetro de línea de comandos, lo que permite pasar de mil a diez mil casillas por lado sin modificar una sola línea de código.
Métricas de evaluación del rendimiento	El sistema corre automáticamente la versión secuencial y la paralela sobre el mismo laberinto, calcula speedup y eficiencia, y exporta el detalle de cada algoritmo a un archivo CSV dentro de la carpeta de métricas.
Aplicación a un problema del mundo real	El mismo esquema de exploración paralela de un grafo grande es el que emplean los sistemas de navegación GPS y los motores de videojuegos para calcular rutas.

Diseño de la solución:

Arquitectura general del sistema

El sistema se organiza en cuatro módulos con responsabilidades claramente separadas. El módulo de generación (Maze) construye la matriz del laberinto en memoria. El módulo de algoritmos (Solvers) contiene las cuatro implementaciones de búsqueda, todas bajo una misma interfaz. El módulo de orquestación (Orchestration) decide cómo y cuándo se ejecuta cada algoritmo, tanto en su versión secuencial como en la paralela. Y el módulo de métricas (Metrics) recoge los resultados de cada ejecución y los deja disponibles en un archivo CSV para su análisis posterior. Program.cs actúa como punto de entrada y conecta estos cuatro módulos en un flujo único: genera el laberinto, ejecuta ambas versiones a través del orquestador, imprime un resumen en consola y exporta el detalle a disco.

Diagrama de componentes



Estrategia de paralelización utilizada:

La decisión de usar `Task.Run` junto con `Task.WaitAll`, en lugar de una construcción como `Parallel.ForEach`, responde a la naturaleza del problema: no se trata de repartir iteraciones idénticas de un bucle entre varios hilos, sino de ejecutar cuatro algoritmos distintos entre sí, cada uno con su propia lógica interna y su propio tiempo de finalización. Es, en otras palabras, paralelismo de tareas heterogéneas, y no paralelismo de datos, por lo que un modelo explícito de tareas resulta más claro y más fiel a lo que realmente ocurre.

Para que las cuatro tareas puedan escribir su resultado sin interferir entre sí, el proyecto utiliza un `ConcurrentDictionary` indexado por el nombre de cada algoritmo. Un diccionario común de .NET no está diseñado para recibir escrituras simultáneas desde varios hilos y podría dejar la estructura en un estado inconsistente si dos tareas escribieran al mismo tiempo; la versión concurrente resuelve ese problema de forma transparente. El laberinto en sí no necesita ningún mecanismo de sincronización adicional, porque una vez generado se trata como una estructura de solo lectura: ningún algoritmo lo modifica durante la búsqueda.

Herramientas y tecnologías empleadas:

- Lenguaje y plataforma: C# sobre .NET 9.
- Paralelismo: Task Parallel Library, mediante `Task.Run` y `Task.WaitAll`.
- Sincronización de datos: `System.Collections.Concurrent.ConcurrentDictionary`.
- Control de versiones: Git y GitHub, con un repositorio público y un flujo de trabajo basado en ramas y Pull Requests.
- Coordinación del equipo: Issues de GitHub para el desglose y seguimiento semanal de tareas.

Implementación técnica:

Estructura del proyecto:

laberinto-paralelo/

— docs/	Documentación: arquitectura, contrato de solvers, estrategia
— src/MazeSolver/	
— Program.cs	Punto de entrada de la aplicación
— Maze/	MazeCell, MazeGenerator
— Solvers/	ISolver, BfsSolver, DfsSolver, AStarSolver, DijkstraSolver
— Orchestration/	SolverOrchestrator (ejecución secuencial y paralela)
— Metrics/	SolveResult, PerformanceTracker
— tests/	Pruebas unitarias del generador y de los solvers
— metrics/	Resultados de las comparativas realizadas

Explicación del código clave:

Todo el sistema gira en torno a una interfaz común, `ISolver`, que cada algoritmo debe implementar. Su forma es deliberadamente simple: recibe el laberinto ya generado junto con la celda de entrada y la de salida, y devuelve un objeto `SolveResult` con el camino encontrado, el tiempo que tardó y la cantidad de nodos que tuvo que visitar. Gracias a este contrato, el orquestador puede tratar a los cuatro algoritmos de manera genérica —como una simple lista de `ISolver`— sin necesidad de conocer cómo funciona cada uno por dentro, lo que además permitió que distintos integrantes del equipo trabajaran sus algoritmos de forma independiente sin pisarse el trabajo.

La generación del laberinto usa `backtracking` aleatorio implementado de forma iterativa, con una pila explícita en lugar de recursión. Esta decisión no es solo de estilo: una implementación recursiva del mismo algoritmo, aplicada sobre un laberinto de mil casillas por lado, puede acumular cientos de miles de llamadas anidadas y agotar la pila de ejecución del programa. La versión iterativa evita ese problema por completo. Además, la generación admite una semilla fija, lo que permite reproducir exactamente el mismo laberinto entre distintas corridas —una condición necesaria para que la comparación entre algoritmos, y entre modo secuencial y paralelo, sea justa.

La orquestación expone dos métodos con el mismo tipo de firma: `RunSequential` recorre la lista de algoritmos uno por uno, ejecutándolos de forma bloqueante; `RunAll` lanza los cuatro con `Task.Run` y espera a que todos terminen con `Task.WaitAll` antes de devolver los resultados. Que ambos métodos compartan la misma forma de entrada y salida es lo que permite medir su diferencia de tiempo de forma directa, simplemente cronometrando cada llamada por separado.

Uso de mecanismos de sincronización:

El único punto del sistema donde varias tareas escriben de forma concurrente es el diccionario de resultados dentro de RunAll. Aunque cada tarea escribe bajo una clave distinta —el nombre de su propio algoritmo— y en la práctica no debería haber colisión entre ellas, se optó por usar una estructura thread-safe de todas formas, en lugar de asumir que las claves nunca van a coincidir. Es una decisión conservadora: prioriza la corrección del programa por sobre un posible ahorro marginal de rendimiento.

Justificación técnica de las decisiones tomadas:

- Backtracking iterativo en el generador, para evitar desbordar la pila de llamadas en laberintos grandes.
- ConcurrentDictionary en lugar de un diccionario común protegido con locks manuales, porque reduce la superficie de error sin sacrificar claridad en el código.
- Semilla fija durante la generación, para que las comparativas entre algoritmos y entre modos de ejecución partan siempre del mismo laberinto.
- Interfaz ISolver compartida entre los cuatro algoritmos, que permite agregar, quitar o modificar un algoritmo sin tocar el orquestador ni el punto de entrada del programa.

Evaluación de desempeño:

Prueba de integración — laberinto de 10x10

Antes de correr el sistema a gran escala, se ejecutó una prueba end-to-end sobre un laberinto pequeño, con el único propósito de confirmar que todas las piezas —generación, los cuatro algoritmos, orquestación y exportación de métricas— funcionan correctamente integradas.

Algoritmo	Camino encontrado	Nodos visitados	Longitud del camino	Tiempo (ms)
DFS	Sí	66	31	0
BFS	Sí	38	31	0
Dijkstra	Sí	38	31	0
A*	Sí	35	31	0

Los cuatro algoritmos encontraron el mismo camino, de longitud 31, lo cual es el comportamiento esperado: el generador construye un laberinto perfecto, sin ciclos, en el que solo existe un camino posible entre la entrada y la salida. La diferencia entre algoritmos aparece en cuánto tuvieron que explorar para encontrarlo: A* fue el más selectivo, con 35 nodos visitados, seguido de cerca por BFS y Dijkstra con 38 cada uno, mientras que DFS visitó 66 nodos por su tendencia a seguir un solo camino hasta el final antes de retroceder.

Comparativa a gran escala

Se ejecutó el sistema completo sobre laberintos de 1,000, 2,000 y 5,000 casillas por lado, midiendo en cada caso el tiempo secuencial, el tiempo paralelo, el speedup y la eficiencia obtenidos.

Tamaño	Tiempo secuencial	Tiempo paralelo	Speedup	Eficiencia
1,000 x 1,000	601 ms	275 ms	2.19x	54.6%
2,000 x 2,000	1,936 ms	976 ms	1.98x	49.6%
5,000 x 5,000	41,515 ms	22,462 ms	1.85x	46.2%

El detalle por algoritmo en la corrida de 5,000 resulta particularmente revelador:

Algoritmo	Nodos visitados	Tiempo (ms)
DFS	14,741,740	1,534
BFS	20,558,486	2,486
Dijkstra	20,558,477	21,532
A*	20,512,130	21,503

Análisis de cuellos de botella y limitaciones:

El resultado más llamativo de estas pruebas no es que el sistema sea más rápido en paralelo (lo es, en los tres tamaños), sino que el speedup y la eficiencia disminuyen a medida que el laberinto crece, en lugar de aumentar. A primera vista podría interpretarse como un problema del paralelismo, pero el detalle por algoritmo muestra la causa real: BFS y DFS resuelven el laberinto de 5,000 en 2.5 y 1.5 segundos respectivamente, mientras que A* y Dijkstra tardan más de 21 segundos cada uno, casi el mismo tiempo entre sí.

Como las cuatro tareas se ejecutan en paralelo pero no tienen una duración comparable, el tiempo total de la ejecución paralela no puede ser menor que el tiempo de la tarea más lenta. En la corrida de 5,000, ese límite teórico es de aproximadamente 21,532 milisegundos, y el tiempo paralelo medido (22,462 milisegundos) está muy cerca de ese límite. En otras palabras, el paralelismo está funcionando correctamente: BFS y DFS terminan casi de inmediato y quedan esperando sin ningún costo adicional, mientras A* y Dijkstra ocupan la mayor parte del tiempo total. El techo del speedup no lo impone la cantidad de núcleos disponibles, sino el desbalance de carga entre las cuatro tareas que se están paralelizando. Es un caso concreto de una limitación conocida del paralelismo de tareas heterogéneas: el beneficio de correr varias tareas a la vez está acotado por la más lenta de ellas.

Se intentó además una corrida con un laberinto de 10,000 por 10,000 casillas —cien millones de nodos—, la cual no llegó a completarse. El registro del kernel del sistema operativo confirma la causa exacta: el proceso llegó a solicitar un total de memoria virtual de aproximadamente 273 gigabytes y a ocupar cerca de 11.4 gigabytes de memoria física real antes de que el propio sistema operativo lo terminara para proteger la estabilidad general del equipo (mecanismo conocido como OOM killer, por out of memory killer). Esto se documenta como una limitación real y conocida del diseño actual: el sistema mantiene el laberinto completo y el estado de cada uno de los cuatro algoritmos en memoria de forma simultánea, lo cual no escala de forma indefinida. Una mejora futura razonable sería liberar o comprimir estructuras intermedias a medida que cada algoritmo concluye, en lugar de conservarlas todas hasta el final de la ejecución, o bien representar el laberinto con una estructura más compacta que un objeto independiente por cada celda.

Trabajo en equipo:

Descripción del reparto de tareas:

El trabajo se dividió en paquetes independientes desde el inicio del proyecto, buscando que cada integrante pudiera avanzar su parte sin depender del trabajo de los demás salvo en los puntos de integración estrictamente necesarios.

Integrante	Responsabilidad principal
Roberto Antonio López García (líder)	Estructura base del repositorio, generador de laberinto, orquestador paralelo e integración final del sistema completo
Esmil Adames	Implementación y pruebas de los algoritmos BFS y DFS
Oscar Disla	Implementación y pruebas de los algoritmos A* y Dijkstra
Mirelys De La Rosa De La Rosa	Módulo de métricas, cálculo de speedup y eficiencia, exportación de resultados y pruebas del generador

El equipo comenzó el proyecto con cinco integrantes. Gabriel Villa Cordero, originalmente responsable del orquestador paralelo, retiró la asignatura durante el desarrollo del proyecto. Su parte fue reasignada al líder del equipo, quien la completó junto con la integración final, sin que esto representara un atraso significativo en el cronograma general.

Herramientas utilizadas para la coordinación

La coordinación del equipo se apoyó en Git y GitHub como eje central: cada aporte se desarrolló en una rama separada y se integró a la rama principal mediante un Pull Request, lo que dejó un historial claro de quién hizo qué y cuándo. Las tareas de cada semana se desglosaron como Issues individuales dentro del repositorio, asignados a cada integrante, y la comunicación del día a día —dudas puntuales, avisos, coordinación de horarios— se manejó por mensajería de grupo.

Conclusiones:

Principales aprendizajes técnicos

El desarrollo del proyecto permitió al equipo trabajar de forma práctica con Task Parallel Library para coordinar tareas heterogéneas, entender por qué una estructura como ConcurrentDictionary es necesaria en cuanto más de un hilo escribe sobre los mismos datos, y comprobar en carne propia cómo el diseño de un contrato común —la interfaz ISolver— simplifica enormemente la integración de piezas desarrolladas por distintas personas en paralelo, tanto en el sentido de cómputo como en el sentido de trabajo en equipo.

Retos enfrentados y superados

El reto más significativo fue de índole organizativa: la salida de un integrante a mitad de proyecto obligó a reasignar una parte completa del sistema —el orquestador paralelo— sin contar con tiempo adicional. También se presentaron dificultades típicas de un primer proyecto colaborativo con Git: cambios en nombres de carpetas y archivos que no coincidían entre sistemas operativos, y al menos un caso en que un cambio se subió directamente a la rama principal sin pasar por revisión, lo que se corrigió revirtiendo el cambio y reintegrándolo correctamente a través de un Pull Request. Distintos miembros del equipo tenían además versiones distintas del SDK de .NET instaladas localmente, lo que se resolvió ajustando el proyecto a una versión compatible con todos.

En la etapa de pruebas a gran escala surgió un reto adicional, de carácter técnico: al intentar correr el sistema sobre un laberinto de 10,000 por 10,000 casillas, el equipo de desarrollo se quedó sin memoria disponible y el sistema operativo terminó procesos abiertos para liberar recursos. Lejos de ser un contratiempo menor, este hallazgo terminó siendo uno de los aportes más valiosos del proceso de pruebas: expuso una limitación real de diseño —el uso de memoria crece de forma proporcional al tamaño del laberinto multiplicado por la cantidad de algoritmos ejecutándose a la vez— que de otra forma habría pasado inadvertida.

Referencias:

- Microsoft. Task Parallel Library (TPL). Microsoft Learn.
- Microsoft. ConcurrentDictionary Class. Microsoft Learn.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press.
- Buck, J. Maze Generation: Algorithm Recap (Randomized Backtracking / Prim's Algorithm).

Anexos:

Manual de ejecución del sistema

Requiere tener instalado el SDK de .NET (versión 9.0 o compatible). Desde la raíz del repositorio:

```
cd src/MazeSolver
```

```
dotnet run -- 1000
```

El argumento numérico indica el tamaño del laberinto (N por N). Si se omite, se utiliza mil por defecto. Al finalizar, el programa imprime en consola los resultados de cada algoritmo y la comparativa secuencial contra paralela, y exporta el detalle a un archivo CSV dentro de la carpeta de métricas de la compilación.

Pruebas complementarias

El detalle completo de las pruebas realizadas, incluyendo las salidas de consola y los archivos CSV generados en cada corrida, se encuentra documentado dentro de la carpeta metrics/ del repositorio.

Enlace al repositorio

<https://github.com/robertoantoniolopezgarcia/laberinto-paralelo>